

JSEngine Reflection System Guide

리플렉션 시스템 구현 구조와 사용법

작성일: 2026-05-20 / UFUNCTION 미구현 최종 구조 기준

작성일: 2026-05-20

대상: JSEngine 리플렉션 시스템을 사용하는 팀원

이 문서는 현재 프로젝트에 구현된 리플렉션 시스템의 구조와 사용법을 설명한다. 현재 시스템은 `UCLASS`, `USTRUCT`, `UENUM`, `UPROPERTY`, `GENERATED\_BODY`를 지원한다. `UFUNCTION`은 구현하지 않기로 결정했다.

1. 개요

현재 리플렉션 시스템의 목표는 다음과 같다.

- 기존 `DECLARE\_CLASS`, `DEFINE\_CLASS`, `FTypeInfo`, `s\_TypeInfo` 기반 RTTI를 제거하고 `UClass` 기반 RTTI로 통합한다.
- `UPROPERTY`가 붙은 멤버 변수를 자동으로 Details 패널, Serialize, Duplicate 흐름에서 사용할 수 있게 한다.
- 빌드 전에 `Scripts/GenerateReflection.py`가 헤더를 스캔하여 `\*.generated.h`와 `Reflection.generated.cpp`를 생성한다.
- C++ 선언 문법은 Unreal 스타일에 가깝게 유지한다.
- `REGISTER\_FACTORY`를 직접 쓰지 않고, generated cpp에서 factory 등록 코드를 자동 생성한다.

2. 리플렉션 계층도

현재 계층은 다음과 같다.

```
UField
├── UStruct
│   ├── UClass
│   └── UScriptStruct
├── UEnum
└── FProperty
    ├── FBoolProperty
    ├── FIntProperty
    ├── FFloatProperty
    ├── FVectorProperty
    ├── FVector4Property
    ├── FStringProperty
    ├── FNameProperty
    ├── FColorProperty
    ├── FObjectProperty
    ├── FSceneComponentProperty
    ├── FStructProperty
    ├── FEnumProperty
    └── FArrayProperty
```

타입	역할
`UField`	모든 리플렉션 필드의 최상위 타입. 이름을 가진다.
`UStruct`	프로퍼티 배열, 크기, 프로퍼티 순회, Serialize 공통 기능을 가진다.
`UClass`	`UObject` 파생 클래스용 메타 타입. 부모 `UClass`와 `IsA`를 가진다.
`UScriptStruct`	`USTRUCT()`로 선언한 C++ struct용 메타 타입.
`UEnum`	`UENUM()`으로 선언한 enum class의 이름과 값 목록을 가진다.
`FProperty`	멤버 변수 하나의 리플렉션 메타. offset, 표시 이름, serialize 이름, 타입 정보를 가진다.

주요 파일은 다음과 같다.

파일	역할
`JSEngine/Source/Engine/Reflection/Field.h`	`UField` 정의
`JSEngine/Source/Engine/Reflection/Struct.h`	`UStruct`, `UScriptStruct` 정의
`JSEngine/Source/Engine/Reflection/Class.h`	`UClass` 정의
`JSEngine/Source/Engine/Reflection/Enum.h`	`UEnum`, `FEnumValue` 정의
`JSEngine/Source/Engine/Reflection/Property.h`	`FProperty`와 하위 property 타입 정의
`JSEngine/Source/Engine/Reflection/Reflection.h`	`UCLASS`, `UPROPERTY`, `GENERATED_BODY` 등 매크로 정의
`JSEngine/Source/Engine/Reflection/Reflection.cpp`	프로퍼티 순회, Details descriptor 생성, Serialize 구현
`Scripts/GenerateReflection.py`	리플렉션 코드 생성기
`JSEngine/Source/Generated/*.generated.h`	헤더별 `GENERATED_BODY` 확장 코드
`JSEngine/Source/Generated/Reflection.generated.cpp`	모든 class, struct, enum, property, factory 등록 코드

3. 코드 생성 흐름

`JSEngine.vcxproj`의 PreBuild step이 빌드 전에 다음 명령을 실행한다.

```
cd /d "$(SolutionDir)" && Scripts\python\python.exe Scripts\GenerateReflection.py
```

`GenerateReflection.py`는 `JSEngine/Source` 아래의 모든 `.h` 파일을 스캔한다.

1. `UCLASS(...)` class 선언을 찾는다.
2. `USTRUCT(...)` struct 선언을 찾는다.
3. `UENUM(...)` enum class 선언을 찾는다.
4. 각 class/struct 내부의 `UPROPERTY(...)` 다음 한 줄 멤버 선언을 파싱한다.
5. 헤더별 `\*.generated.h` 파일을 생성한다.
6. 전체 등록 코드를 `Reflection.generated.cpp` 하나에 생성한다.

예를 들어 `CameraComponent.h`에 `UCameraComponent`, `FCameraState`가 있으면 다음 파일이 생성된다.

```
JSEngine/Source/Generated/CameraComponent.generated.h
JSEngine/Source/Generated/Reflection.generated.cpp
```

`\*.generated.h`는 `GENERATED\_BODY()`가 실제로 확장될 매크로를 가진다.

```
#define Engine_Component_CameraComponent_h_67_GENERATED_BODY \
public: \
    friend const UClass* Z_Construct_UClass_UCameraComponent(); \
    using ThisClass = UCameraComponent; \
    using Super = USceneComponent; \
    static const UClass* StaticClass(); \
    const UClass* GetClass() const override;
```

`USTRUCT`의 generated body는 다음처럼 `StaticStruct()` 선언을 만든다.

```
#define Engine_Component_CameraComponent_h_15_GENERATED_BODY \
public: \
    static const UScriptStruct* StaticStruct();
```

`Reflection.generated.cpp`에는 다음 코드가 생성된다.

- `Z\_Construct\_UClass\_XXX()`
- `ClassName::StaticClass()`
- `ClassName::GetClass()`
- `Z\_Construct\_UScriptStruct\_XXX()`
- `StructName::StaticStruct()`
- `Z\_Construct\_UEnum\_XXX()`
- 정적 `FProperty` 객체와 property 배열

- 'FObjectFactory' 자동 등록 코드

중요: generated 파일은 직접 수정하지 않는다. 수정이 필요하다면 원본 헤더 또는 'GenerateReflection.py'를 수정한다.

4. 메타데이터와 offset 기반 접근

각 'UClass', 'UScriptStruct', 'UEnum'은 'Reflection.generated.cpp' 안에서 function-local static 객체로 만들어진다. 이 static 객체의 주소가 런타임 타입 식별자로 쓰인다.

```
const UClass* UCameraComponent::StaticClass()
{
    return Z_Construct_UClass_UCameraComponent();
}

const UClass* UCameraComponent::GetClass() const
{
    return UCameraComponent::StaticClass();
}
```

'UClass'는 다음 정보를 가진다.

- 클래스 이름
- 부모 클래스 'SuperClass'
- 'sizeof(ClassName)'
- 해당 클래스에 선언된 'FProperty' 배열
- property 개수

'IsA'는 'SuperClass' 체인을 따라 올라가며 'UClass' 포인터 주소를 비교한다. 'Cast<T>'와 'UObject::IsA<T>()'도 이 구조를 사용한다.

프로퍼티는 멤버 변수의 실제 주소를 저장하지 않는다. 대신 클래스 또는 구조체 시작 주소로부터의 offset을 저장한다.

```
static const FFloatProperty Property_FieldOfView(
    "FieldOfView",
    "FOV",
    nullptr,
    offsetof(FCameraState, FieldOfView),
    EPropertyAccess::EditAnywhere,
    0.1f,
    3.14f,
    0.01f,
    EPropertyUsageFlags::Animatable);
```

실제 값 접근은 다음 방식이다.

```
void* ValuePtr = static_cast<uint8*>(Container) + Offset;
```

따라서 같은 'FProperty' 메타데이터를 여러 인스턴스에 공통으로 사용할 수 있다.

5. UPROPERTY 지원 범위

기본 문법은 다음과 같다.

```
UPROPERTY(EditAnywhere)
float BlendAlpha = 0.0f;

UPROPERTY(VisibleAnywhere, DisplayName="Current Time")
float CurrentTime = 0.0f;

UPROPERTY(EditAnywhere, DisplayName="FOV", Min=0.1, Max=3.14, Speed=0.01, Animatable)
float FieldOfView = 1.047f;
```

첫 번째 인자는 반드시 'EditAnywhere' 또는 'VisibleAnywhere'이다.

지정자	Details 표시	Details 수정	Serialize
`EditAnywhere`	예	예	예
`VisibleAnywhere`	예	아니오	예

추가 metadata는 다음을 지원한다.

메타	의미
`DisplayName="..."`	Details 패널 표시 이름. `SerializeName`이 없으면 저장 key로도 사용된다.
`SerializeName="..."`	Serialize key를 별도로 지정한다.
`Min=...`	숫자 위젯 최소 힌트
`Max=...`	숫자 위젯 최대 힌트
`Speed=...`	DragFloat 등 UI 조작 속도
`Animatable`	애니메이션 가능한 값 표시 플래그
`Transient` 또는 `NonSerialized`	자동 Serialize 제외
`Type=SceneComponentRef`	Actor 내부 SceneComponent 참조 방식으로 처리

Serialize key 우선순위는 다음과 같다.

```
SerializeName ■ ■ ■ ■ SerializeName
■ ■ ■ ■ DisplayName
■ ■ ■ ■ ■ ■ ■ ■ ■ ■
```

현재 지원 타입은 다음과 같다.

C++ 타입	FProperty	EPropertyType
`bool`	`FBoolProperty`	`Bool`
`int`, `int32`	`FIntProperty`	`Int`
`float`	`FFloatProperty`	`Float`
`FVector`	`FVectorProperty`	`Vec3`
`FVector4`	`FVector4Property`	`Vec4`
`FColor`	`FColorProperty`	`Color`
`FString`	`FStringProperty`	`String`
`FName`	`FNameProperty`	`Name`
`UObject` 파생 포인터	`FObjectProperty`	`ObjectRef`
`USTRUCT` 타입	`FStructProperty`	`Struct`
`UENUM` 타입	`FEnumProperty`	`Enum`
`TArray<FVector>`	`FArrayProperty`	`Vec3Array`

제한 사항: `UPROPERTY` 다음 멤버 선언은 한 줄이어야 한다.

```
UPROPERTY(EditAnywhere)
float Value = 0.0f; // ■■
```

복잡한 여러 줄 선언, 임의 템플릿, 임의 배열 타입은 지원하지 않는다.

6. UCLASS 사용법

`UObject` 파생 클래스는 다음 규칙을 따른다.

```
#pragma once

#include "Object/Object.h"
#include "Generated/MyComponent.generated.h"

UCLASS()
```

```
class UMyComponent : public UActorComponent
{
public:
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, DisplayName="Move Speed", Min=0.0, Max=3000.0, Speed=1.0)
    float MoveSpeed = 600.0f;
};
```

필수 규칙은 다음과 같다.

1. 클래스 선언 바로 위에 `UCLASS(...)`를 붙인다.
2. 클래스 본문 안 `public` 영역에 `GENERATED\_BODY()`를 넣는다.
3. 해당 헤더는 `Generated/<원본파일명>.generated.h`를 include한다.
4. generated include는 `GENERATED\_BODY()`를 사용하기 전에 있어야 한다.
5. generated 파일은 직접 수정하지 않는다.

`UCLASS(Abstract)`는 리플렉션 메타는 생성하지만 `FObjectFactory` 자동 생성 대상에서는 제외한다.

```
UCLASS(Abstract)
class UMovementComponent : public UActorComponent
{
public:
    GENERATED_BODY()
};
```

`Abstract`는 C++ 순수 가상 여부와 별개로 "이 UClass를 직접 생성하지 않는다"는 리플렉션 메타로 사용한다.

`UCLASS()`로 등록된 concrete 클래스는 `Reflection.generated.cpp`에서 자동으로 `FObjectFactory`에 등록된다. 따라서 `REGISTER\_FACTORY`는 사용하지 않는다.

7. USTRUCT / UScriptStruct 사용법

C++ struct는 `USTRUCT()`로 선언하고 `GENERATED\_BODY()`를 넣는다. 런타임 메타 타입은 `UScriptStruct`이다.

```
#include "Generated/CameraComponent.generated.h"

USTRUCT()
struct FCameraState
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, DisplayName="FOV", Min=0.1, Max=3.14, Speed=0.01, Animatable)
    float FieldOfView = 1.047f;

    UPROPERTY(EditAnywhere, DisplayName="Near Z", SerializeName="NearClip")
    float NearClip = 0.1f;
};
```

다른 `UPROPERTY`에서 `USTRUCT` 타입을 멤버로 쓰면 `FStructProperty`가 생성된다. `FStructProperty`는 내부 `UScriptStruct`의 프로퍼티를 다시 순회한다.

```
UCLASS()
class UCameraComponent : public USceneComponent
{
public:
    GENERATED_BODY()

    UPROPERTY(EditAnywhere)
    FCameraState CameraState;
};
```

8. UENUM 사용법

`enum class`는 `UENUM()`을 붙이면 리플렉션된다.

```
UENUM()
enum class EInterpBehaviour : uint8
```

```
{
    OneShot UMETA(DisplayName="One Shot"),
    Loop UMETA(DisplayName="Loop"),
};
```

`UPROPERTY`로 enum 멤버를 선언하면 `FEnumProperty`가 생성된다.

```
UPROPERTY(EditAnywhere, DisplayName="Behaviour")
EInterpBehaviour Behaviour = EInterpBehaviour::OneShot;
```

주의: enum 값 중 `MAX`, `Max`, `Count`, `COUNT`, `Num`, `NUM`은 sentinel로 보고 제외한다.

9. Details 패널과 Serialize 흐름

`UObject`는 공통으로 다음 흐름을 제공한다.

```
void AppendReflectedProperties(TArray<FPropertyDescriptor>& OutProps);
void GetAllEditableProperties(TArray<FPropertyDescriptor>& OutProps);
void SerializeReflectedProperties(FArchive& Ar);
```

`GetAllEditableProperties`는 다음 순서로 동작한다.

1. 기존 수동 `GetEditableProperties(OutProps)`를 호출한다.
2. `AppendReflectedProperties(OutProps)`를 호출한다.

따라서 새 일반 멤버 변수는 `GetEditableProperties`에 수동 추가하지 말고 `UPROPERTY`로 등록한다. 다만 Material, SRV, CubeSRV, Lua script 동적 프로퍼티처럼 아직 `UPROPERTY`로 표현하지 못하는 값은 수동 등록으로 남을 수 있다.

Serialize는 `UObject::Serialize`에서 공통으로 다음을 호출한다.

```
SerializeReflectedProperties(Ar);
```

각 `FProperty`는 자신의 `SerializeItem`을 통해 `FArchive`에 값을 저장하거나 로드한다. `Transient` 또는 `NonSerialized`가 붙은 프로퍼티는 자동 Serialize에서 제외된다.

10. ObjectRef와 SceneComponentRef

`UObject` 파생 포인터 멤버는 `ObjectRef`로 등록된다.

```
UPROPERTY(EditAnywhere, DisplayName="AnimInstance")
UAnimSingleNodeInstance* SingleNodeInstance = nullptr;
```

코드젠은 포인터 타입을 보고 다음처럼 `ObjectType`을 기록한다.

```
UAnimSingleNodeInstance::StaticClass()
```

`FObjectProperty`는 Serialize 시 객체 포인터 자체가 아니라 UUID를 저장한다. 로드 시 `GUObjectArray`를 순회하여 같은 UUID를 가진 `UObject`를 다시 연결한다.

`SceneComponentRef`는 일반 `ObjectRef`와 다르게 Actor 내부 component 참조를 복원하기 위한 특수 타입이다.

```
UPROPERTY(EditAnywhere, DisplayName="Updated Component", Type=SceneComponentRef)
USceneComponent* UpdatedComponent = nullptr;
```

`SceneComponentRef`는 로드 시 owner actor의 components 배열에서 UUID가 같은 `USceneComponent`를 찾는다.

11. Factory 자동 등록

`FObjectFactory`는 클래스 이름 문자열로 `UObject`를 생성하는 레지스트리이다. 이제 `REGISTER\_FACTORY` 매크로는 사용하지 않는다.

`Reflection.generated.cpp`가 concrete `UCLASS`에 대해 자동 등록 코드를 만든다.

```
FObjectFactory::Get().Register(
    "UAnimNotify_PlaySFX",
    []() -> UObject* { return UObjectManager::Get().CreateObject<UAnimNotify_PlaySFX>(); },
    UAnimNotify_PlaySFX::StaticClass());
```

‘UCLASS(Abstract)’가 붙은 클래스는 리플렉션 메타는 생성되지만 factory 등록은 제외된다.

에디터에서 생성 가능한 클래스 목록이 필요하면 다음 API를 사용한다.

```
TArray<const UClass*> Classes;
FObjectFactory::Get().GetRegisteredClasses(Classes);
```

클래스 호환성 검사는 ‘UClass::IsA’를 사용한다.

```
if (Class->IsA(UAnimNotifyState::StaticClass()))
{
    // NotifyState ■■
}
```

12. 사용 예시와 체크리스트

새 reflected component 예시는 다음과 같다.

```
#pragma once

#include "Component/ActorComponent.h"
#include "Generated/MyMovementComponent.generated.h"

UCLASS()
class UMyMovementComponent : public UActorComponent
{
public:
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, DisplayName="Move Speed", Min=0.0, Max=3000.0, Speed=1.0, Animatable)
    float MoveSpeed = 600.0f;

    UPROPERTY(VisibleAnywhere, DisplayName="Current Velocity")
    FVector CurrentVelocity = FVector::ZeroVector;
};
```

새 reflected struct 예시는 다음과 같다.

```
#pragma once

#include "Generated/MyCameraData.generated.h"

USTRUCT()
struct FMyCameraData
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, DisplayName="FOV", Min=0.1, Max=3.14, Speed=0.01)
    float FOV = 1.047f;
};
```

새 reflected enum 예시는 다음과 같다.

```
UENUM()
enum class EWeaponMode : uint8
{
    Single UMETA(DisplayName="Single"),
    Burst UMETA(DisplayName="Burst"),
    Auto UMETA(DisplayName="Auto"),
};
```

체크리스트는 다음과 같다.

- ‘UCLASS’, ‘USTRUCT’, ‘UENUM’ 중 필요한 매크로를 붙였는가?
- ‘GENERATED_BODY()’를 넣었는가?
- ‘Generated/<파일명>.generated.h’를 include했는가?

- 'UPROPERTY' 다음 멤버 선언이 한 줄인가?
- 지원 타입만 'UPROPERTY'로 등록했는가?
- 직접 생성되면 안 되는 클래스에 'UCLASS(Abstract)'를 붙였는가?
- generated 파일을 직접 수정하지 않았는가?

13. 현재 제한 사항

현재 의도적으로 지원하지 않거나 수동 처리로 남겨둔 영역은 다음과 같다.

- 'UFUNCTION'은 구현하지 않는다.
- Material, SRV, CubeSRV 같은 렌더 리소스 전용 프로퍼티는 수동 'GetEditableProperties'가 필요할 수 있다.
- Lua script 동적 프로퍼티는 'UPROPERTY'가 아니라 런타임 동적 시스템으로 유지한다.
- Actor transform처럼 실제 멤버가 아니라 계산/적용 로직이 있는 값은 수동 노출이 필요할 수 있다.
- 임의 배열, 임의 템플릿, 복잡한 멀티라인 선언은 'GenerateReflection.py'가 파싱하지 않는다.
- GC는 아직 구현하지 않았다. 다만 'UField', 'UClass', 'FProperty' 구조는 향후 GC를 얻을 수 있는 방향이다.

새 코드에서는 'DECLARE_CLASS', 'DEFINE_CLASS', 'REGISTER_FACTORY', 'FTypeInfo', 's_TypeInfo'를 사용하지 않는다.